

CM 2005 Object Oriented Programming

Final Assignment Report

Name: Dhanarasu Naveen

SIM Student ID: 10250211

UOL Student ID: 230655533

Table of Contents :

R1: Basic Functionality	4
R1A: Loading Audio Files into Audio Players.....	4
Overview	4
Implementation Details.....	4
1. Loading via File Chooser	4
2. Loading from the Playlist Component	5
Summary for R1A.....	6
R1B: Playing Two or More Tracks Simultaneously	6
Overview	6
Implementation Details.....	6
R1C: Mixing the Tracks by Varying Their Volumes	7
Overview	7
Implementation Details.....	7
Summary for R1C	8
R1D: Speeding Up and Slowing Down the Tracks.....	8
Overview	8
Implementation Details.....	9
Summary for R1D	9
Overall Summary of R1	10
R2: Customising the User Interface	10
R2A: Significantly Different Layout.....	10
Inspiration and Concept	10
Implementation Highlights.....	12
Final Design	13
Differences from the Basic DeckGUI	13
R2B: New Event Listeners	13
1. Drum Pad Double-Click Listener.....	13
2. Jog Wheel Rotation Callback	14
Summary of R2.....	15
R3: Implementation of New DJ Features	15
Drum Pads	15
Research Background.....	15
Purpose and Functionality.....	16
Code Snippets for Drum Pads	16
Drum Pad 1 & Drum Pad 6 – Loading & Reloading Samples	16
Drum Pad 2 – Bass Drop Effect.....	18
Drum Pad 3 – Kick Roll Effect.....	19
Drum Pads 4 & 5 – Beat Skipping.....	20

Jog Wheel	20
Purpose and Functionality.....	20
How It Works.....	20
Three-Band EQ (Low, Mid, High)	21
Purpose and Functionality.....	21
Research Material	21
Record Button	23
Purpose and Functionality.....	23
How It Works.....	23
VU Meter	24
Purpose and Functionality.....	24
How It Works.....	24
Summary of R3.....	25

R1: Basic Functionality

The Otodecks DJ application implements the fundamental features expected in a DJ software. In this section, we detail how the application meets each of the following basic requirements:

- **R1A:** Loading audio files into audio players
- **R1B:** Playing two or more tracks simultaneously
- **R1C:** Mixing the tracks by varying their volumes
- **R1D:** Speeding up and slowing down the tracks

Each subsection provides an in-depth explanation of the code structure, design decisions, and relevant code excerpts.

R1A: Loading Audio Files into Audio Players

Overview

The application allows users to load audio files into the audio players through two primary methods:

1. **Direct File Chooser:** Users can load audio files directly into a deck using a file chooser dialog.
2. **Playlist Component:** Users can also select and load tracks from a persisted music library managed by the playlist component.

This dual approach enhances usability by providing flexibility in how tracks are loaded.

Implementation Details

1. Loading via File Chooser

- **User Interaction:**
When the user clicks the "Load" button associated with a deck, the application triggers a file chooser dialog.
- **Code Flow:**
In **MainComponent.cpp**, the **buttonClicked** method launches the file chooser asynchronously. If a valid file is selected, it is wrapped into a **juce::URL** and passed to the deck's **loadTrack** method.

```
else if (button == &loadButton1)
{
    auto flags = juce::FileBrowserComponent::canSelectFiles;
    fChooser.launchAsync(flags, [this] (const juce::FileChooser& fc)
    {
        if (fc.getResult().existsAsFile())
            deckGUI1.loadTrack(juce::URL{ fc.getResult() });
    }); }
```

- **Deck Integration:**

Within the **DeckGUI** class, the **loadTrack** method is responsible for forwarding the audio file URL to the **DJAudioPlayer** for playback as well as updating the waveform display.

```
void DeckGUI::loadTrack(juce::URL audioURL)
{
    player->loadURL(audioURL);
    waveformDisplay.loadURL(audioURL);
}
```

- **Audio Player Initialization:**

The **DJAudioPlayer::loadURL** method handles the conversion of the file into an audio stream, ensuring that the audio data is ready for playback.

```
void DJAudioPlayer::loadURL(juce::URL audioURL)
{
    auto stream = audioURL.createInputStream(false, nullptr, nullptr, {}, 0, nullptr);
    if (stream != nullptr)
    {
        if (auto* reader = formatManager.createReaderFor(std::move(stream)))
        {
            std::unique_ptr<juce::AudioFormatReaderSource> newSource(
                new juce::AudioFormatReaderSource(reader, true)
            );
            transportSource.setSource(newSource.get(), 0, nullptr, reader->sampleRate);
            readerSource.reset(newSource.release());
        }
    }
}
```

2. Loading from the Playlist Component

- **Playlist Functionality:**

In addition to the file chooser, the application features a playlist component that manages a library of audio tracks. This library is persisted between application sessions.

- **User Interaction:**

Users can search through the playlist and select a track to load into the decks. The playlist component provides a UI for adding, filtering, and selecting tracks.

- **Integration with Decks:**

The playlist component includes callback functions (e.g., **loadDeck1Callback** and **loadDeck2Callback**) that, when triggered, pass the chosen track's URL to the respective deck's **loadTrack** method.

```
playlistComponent.loadDeck1Callback = [this](const juce::URL& url)
{
    deckGUI1.loadTrack(url);
};
playlistComponent.loadDeck2Callback = [this](const juce::URL& url)
{
    deckGUI2.loadTrack(url);
};
```

This callback mechanism ensures seamless integration between the persistent playlist and the live decks, allowing users to quickly load tracks from their music library.

Summary for R1A

The Otodecks DJ application successfully implements the requirement to load audio files into audio players by offering two distinct methods:

- **Direct Loading:** A file chooser dialog in the deck interface allows users to load audio files directly into a deck.
- **Playlist Integration:** A dedicated playlist component manages a persistent music library and provides callbacks to load selected tracks into the decks.

Both methods ensure that audio files are properly processed by the **DJAudioPlayer** and prepared for playback, fulfilling the R1A requirement.

R1B: Playing Two or More Tracks Simultaneously

Overview

Simultaneous playback of multiple tracks is essential in DJ applications. Otodecks achieves this by utilizing separate audio player instances for each deck, which are then combined into a single audio output stream using a mixer.

Implementation Details

1. Multiple Audio Player Instances:

- **Creation of Players:**

In **MainComponent.cpp**, two primary instances of **DJAudioPlayer** (**player1** and **player2**) are instantiated. Each instance is associated with its own **DeckGUI** (i.e., **deckGUI1** and **deckGUI2**).

- **Dedicated Drum Players:**

Additionally, there are drum players (**drumPlayer1** and **drumPlayer2**) used for triggering special effects like bass drops and kick rolls.

2. Audio Mixing:

- **MixerAudioSource Usage:**

The audio outputs from all players are combined using a **MixerAudioSource** object. This mixer takes multiple input sources and blends them into a single output stream that is delivered to the audio device.

```
mixerSource.addInputSource(&player1, false);
mixerSource.addInputSource(&player2, false);
mixerSource.addInputSource(&drumPlayer1, false);
mixerSource.addInputSource(&drumPlayer2, false);
```

The mixer ensures that even when multiple tracks are played at once, their audio signals are appropriately merged.

3. Simultaneous Playback:

- **Independent Control:**

Each deck can start, stop, or adjust playback independently. For example, the "Play" button in each deck is linked to the **start()** method of its respective **DJAudioPlayer**.

```
if (button == &playButton1) player1.start();
else if (button == &playButton2) player2.start();
```

Summary for R1B

By creating multiple instances of **DJAudioPlayer** and routing them through a **MixerAudioSource**, the application supports the simultaneous playback of several tracks. This design allows for independent control over each track while ensuring that all audio outputs are mixed together in real time.

R1C: Mixing the Tracks by Varying Their Volumes

Overview

A crucial aspect of DJ mixing is controlling the volume of each track. Otodecks implement this through volume sliders that adjust the gain of each audio player, enabling users to balance the mix.

Implementation Details

1. Volume Control via Sliders:

- **UI Components:**

Each deck has a dedicated volume slider (**volSlider1** for Deck 1 and **volSlider2** for Deck 2) displayed in the user interface.

- **Event Handling:**

The **sliderValueChanged** method in **MainComponent.cpp** captures changes in the slider's value and passes the new value to the corresponding audio player's **setGain** method.

```
if (slider == &volSlider1) player1.setGain(slider->getValue());  
else if (slider == &volSlider2) player2.setGain(slider->getValue());
```

This event listener ensures that any adjustment made by the user immediately affects the playback volume.

2. Audio Player Gain Adjustment:

- **Method Implementation:**

In **DJAudioPlayer.cpp**, the **setGain** function adjusts the gain of the **transportSource**:

```
void DJAudioPlayer::setGain(double gain)  
{  
    if (gain < 0.0 || gain > 1.0)  
        DBG("DJAudioPlayer::setGain out of [0..1] range!");  
    else  
        transportSource.setGain(gain);  
}
```

This method ensures that the gain value is within a valid range and then applies it to the audio transport source.

3. Mixing through the Mixer:

- **Combined Output:**

Because each player's output is routed through the **MixerAudioSource**, the individual volume adjustments directly affect the overall mix heard by the user. This design provides precise control over the relative loudness of each track.

Summary for R1C

The application successfully implements track mixing by using volume sliders to control the gain of each audio player. The real-time adjustments made through the UI directly impact the mixed output produced by the mixer, allowing for a balanced audio experience.

R1D: Speeding Up and Slowing Down the Tracks

Overview

Controlling the playback speed of a track is a key feature in DJ applications. This allows DJs to match tempos and creatively manipulate tracks. Otodecks enable speed control through resampling.

Implementation Details

1. Speed Control via Sliders:

- **UI Components:**

Each deck has a dedicated speed slider (**speedSlider1** for Deck 1 and **speedSlider2** for Deck 2). These sliders allow users to adjust the playback speed dynamically.

- **Event Handling:**

Similar to the volume control, the **sliderValueChanged** method detects changes in the speed slider and calls the **setSpeed** method of the corresponding **DJAudioPlayer**.

```
else if (slider == &speedSlider1) player1.setSpeed(slider->getValue());  
else if (slider == &speedSlider2) player2.setSpeed(slider->getValue());
```

2. Resampling and Speed Adjustment:

- **ResamplingAudioSource:**

In the **DJAudioPlayer** class, speed control is achieved by using a **ResamplingAudioSource** that wraps the **transportSource**. The resampling ratio is adjusted to alter the playback speed.

- **Method Implementation:**

The **setSpeed** method ensures that the desired speed ratio is within acceptable limits and then applies it:

```
void DJAudioPlayer::setSpeed(double ratio)  
{  
    if (ratio < 0.01 || ratio > 100.0)  
        DBG("DJAudioPlayer::setSpeed out of [0..100] range!");  
    else  
        resampleSource.setResamplingRatio(ratio);  
}
```

By modifying the resampling ratio, the application effectively speeds up or slows down the audio without the need for pitch correction.

3. Real-Time Feedback:

- **Visual Syncing:**

The waveform display and jog wheel are also updated to reflect changes in playback speed. This provides visual feedback to the user about the current track position and tempo adjustments.

Summary for R1D

Speed control is implemented through speed sliders that adjust the resampling ratio of each audio player. This allows users to dynamically alter the playback speed of the tracks, enabling creative tempo matching and other performance techniques.

Overall Summary of R1

The Otodecks DJ application comprehensively fulfills the basic functionality requirements by:

- **R1A:** Allowing users to load audio files through a file chooser and initializing the audio players.
- **R1B:** Supporting simultaneous playback by using separate audio players and combining their outputs with a mixer.
- **R1C:** Enabling track mixing through individual volume sliders that adjust the gain of each player.
- **R1D:** Allowing dynamic speed control using resampling techniques, with user input via speed sliders.

These core features are implemented using an object-oriented approach, with dedicated classes such as **DJAudioPlayer**, **DeckGUI**, and **MainComponent** ensuring a modular and maintainable design.

R2: Customising the User Interface

R2A: Significantly Different Layout

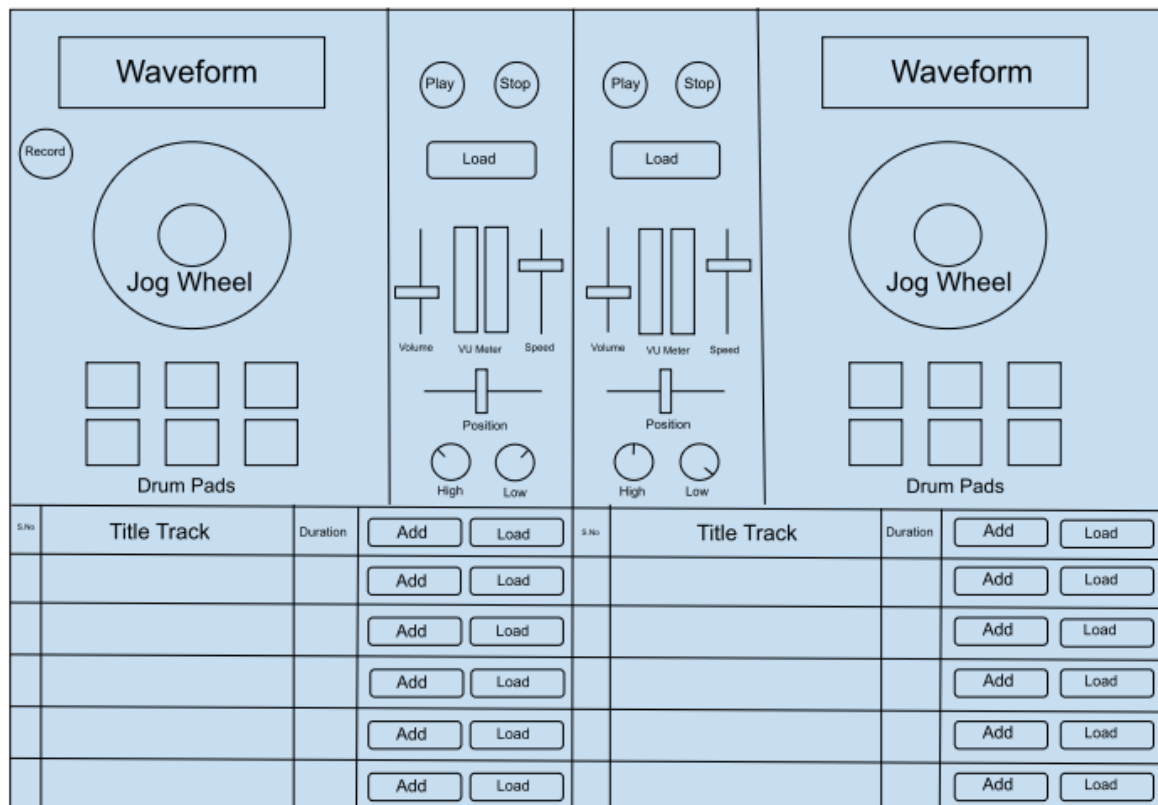
Inspiration and Concept

To move beyond the basic **DeckGUI** shown in class, I explored real DJ controllers online and discovered the **AlphaTheta DDJ-FLX2 Compact 2-Channel DJ Controller**. This hardware layout inspired my final design. Key elements include:



- **Jog Wheels** at the top for track manipulation
- **Performance Pads** or “Drum Pads” beneath each jog wheel
- **Mixer Controls** (Volume, Speed, EQ knobs) arranged centrally
- **Waveform Displays** for visual feedback of track progress
- **Load Buttons** for quick file loading
- **Playlist Section** at the bottom for managing and loading tracks

Below is my conceptual blueprint, illustrating how I translated the hardware layout into a software UI. It contains two decks, each with:



1. A **Waveform** display
2. A **Jog Wheel** component
3. **Drum Pads** for triggering samples and effects
4. Faders/Sliders for **Volume**, **Speed**, and **EQ**
5. **Load** and **Transport** (Play/Stop) buttons
6. **VU Meters** to visualize real-time audio levels
7. A **Record Button** (included on Deck 1)

Additionally, I planned a **Playlist Table** at the bottom for managing and loading tracks.

Note on Blueprint vs. Final Design

- **Search Bar:** Although my initial blueprint did not show a search bar, I added one in the final design for improved filtering within the playlist.

- **Mid EQ Knob:** My blueprint listed only Low and High EQ knobs, but I decided to include a **MidEQ** knob in the final implementation, as real DJ controllers often have three-band EQ.

Implementation Highlights

1. DeckGUI Modifications

- **WaveformDisplay:** Placed at the top for each deck, giving a clear visual of track progress.
- **JogWheel:** Centered in each deck, modeled after hardware jog wheels, complete with custom painting for neon effects.
- **DrumPads:** Positioned in a grid below the jog wheel for quick access to vocal samples, bass drops, or skip controls.
- **EQ Knobs:** Arranged horizontally in each deck for Low, Mid, and High adjustments. The Mid knob was not in the original blueprint but was added later to align with typical DJ gear.
- **Record Button:** On Deck 1, a special toggle button for recording the master output.
- **VU Meters:** Added alongside the volume sliders to provide a visual representation of the left/right channel levels in real time.

2. Playlist Component

- **Expanded Layout:** Occupies the lower portion of the window.
- **Track Table:** Displays columns for track title, duration, and load actions.
- **Search Bar:** Introduced to enable quick filtering of tracks by name (not shown in the original blueprint but included in the final design).
- **Persistence:** Stores the playlist in an XML file so it remains available on subsequent launches.

3. Color Scheme and Theming

- **Neon Glow Aesthetic:** Moved away from default JUCE color schemes in favor of a vibrant palette.
- **Gradients:** Teal, pink, and purple gradients are used in sliders, buttons, and backgrounds.
- **Custom LookAndFeel:** Implemented **RealisticButtonLookAndFeel** and **FancyFaderLookAndFeel** for unique outlines, gradient fills, and rounded buttons.

Final Design



Differences from the Basic DeckGUI

Compared to the minimal controls in the original class code (volume, speed, position slider, and a load button), my new design features:

- **Visually:** Jog wheels, drum pads, waveform expansions, neon-themed backgrounds, VU meters, and a more hardware-inspired deck layout.
- **Functionally:** Adds recording capabilities, custom effect triggers (bass drop, kick roll), skip controls, three-band EQ (including Mid), VU meters for audio level monitoring, and a search bar for the playlist.
- **Layout:** Arranged to mimic real DJ gear, departing from the simple stacked controls of the basic DeckGUI.

R2B: New Event Listeners

In addition to the enhanced layout and additional components, I introduced new event listeners that were not part of the original class codebase. Two prominent examples are:

1. Drum Pad Double-Click Listener

This listener allows the user to double-click on specific drum pads (e.g., “Pad1” or “Pad6”) to trigger a reload of the vocal sample—an interaction not available in the original DeckGUI implementation.

Code Snippet:

```
void DrumPads::DrumPadButton::mouseDoubleClick(const juce::MouseEvent& event)
{
    if (getName() == "Pad1" || getName() == "Pad6")
    {
        if (auto* parent = findParentComponentOfClass<DrumPads>())
            parent->handleDrumPadDoubleClick(getName());
    }
    juce::TextButton::mouseDoubleClick(event);
}
```

Explanation:

- When the user double-clicks on a drum pad named "Pad1" or "Pad6", this overridden method is called.
- The code checks the pad's name and, if it matches, retrieves its parent component (the DrumPads container) and calls a custom handler (**handleDrumPadDoubleClick**) with the pad's name.
- This interaction provides a quick method to reload a sample, extending the basic single-click functionality.

2. Jog Wheel Rotation Callback

The JogWheel component features callbacks (**onJogTurn** and **onJogEnd**) that allow it to respond to mouse drag events by calculating the rotational delta. This mechanism translates the rotational movement of the jog wheel into changes in the playhead position, providing more realistic control.

Code Snippet:

```
void JogWheel::mouseDrag(const juce::MouseEvent& event)
{
    if (isDragging)
    {
        if (!isPointInsideWheel(event.position))
            return;
        float currentAngle = computeAngle(event.position);
        float deltaAngle = currentAngle - lastAngle;
        // Adjust deltaAngle if wrapping around.
        if (deltaAngle > juce::MathConstants<float>::pi)
            deltaAngle -= juce::MathConstants<float>::twoPi;
        else if (deltaAngle < -juce::MathConstants<float>::pi)
            deltaAngle += juce::MathConstants<float>::twoPi;
        lastAngle = currentAngle;
        float deltaPosition = deltaAngle / juce::MathConstants<float>::twoPi;
        playheadPosition = juce::jlimit(0.0f, 1.0f, playheadPosition + deltaPosition);
        if (onJogTurn)
            onJogTurn(deltaPosition);
        repaint();
    }
}
```

Explanation:

- When the user drags the mouse within the jog wheel, the method calculates the change in angle (deltaAngle) based on the current and last recorded angles.
- It adjusts the deltaAngle to account for the circular wrap-around, then computes a normalized deltaPosition.
- The **onJogTurn** callback is invoked with the deltaPosition value, allowing external components (e.g., the DeckGUI) to update the playhead position in real time.
- This rotation-based callback enhances the interactivity, simulating the physical rotation of a DJ jog wheel.

Summary of R2

- **Layout:** Mimics professional DJ gear with waveform displays, jog wheels, drum pads, three-band EQ, VU meters, record toggle, and a searchable playlist.
 - **Custom Look and Feel:** Utilizes neon gradients, rounded buttons, and stylized sliders via JUCE's LookAndFeel system.
 - **New Event Listeners:** Adds double-click handling on drum pads and angle-based jog wheel callbacks for enhanced interactivity.
-

R3: Implementation of New DJ Features

In this extension, I significantly enhanced the Otodecks DJ application by introducing advanced performance tools inspired by professional DJ controllers. My primary focus was on implementing **Drum Pads**, a feature I researched extensively through online resources and videos. In addition to the drum pads, I integrated a Jog Wheel, three-band EQ controls (Low, Mid, High), a Record Button, and VU Meters to create a comprehensive, interactive DJ interface.

Drum Pads

Research Background

To understand how drum pads are used in a live DJ context, I explored various online resources and demonstrations. One influential video was [this demonstration](#), which showed how DJs use drum pads to trigger vocal samples, sound effects, and live performance cues such as bass drops and beat scratching. Additional research included exploring product details of modern DJ controllers and reading online articles about performance pad usage. This research confirmed that drum pads are typically used for:

- Triggering short audio samples (e.g., vocals, sound effects)
- Initiating live effects (e.g., bass drops, kick rolls)
- Navigating tracks with beat skipping (rewind/fast-forward)

Purpose and Functionality

The drum pads in my implementation serve multiple purposes:

- **Drum Pad 1 and Drum Pad 6 – Load and Reload Samples:**
 - **Loading:**
When no sample is loaded, these pads allow the DJ to load vocal tracks or sound effects via a file chooser.
 - **Reloading:**
A double-click on either pad clears the current sample and reopens the file chooser, enabling rapid reloading during performance.
 - **Sample Duration:**
To ensure that these pads function as short, triggerable clips rather than playing an entire song, the application limits playback to the first 10 seconds of the loaded sample.
- **Drum Pad 2 – Bass Drop Effect:**
Rather than loading an external audio file, this pad is hard-coded to trigger a synthesized bass drop effect. Bass drops are commonly used at the end of a track for dramatic impact.
- **Drum Pad 3 – Kick Roll Effect:**
This pad generates a kick roll effect through code, producing a rolling kick drum sound that adds dynamic flair to a live set.
- **Drum Pads 4 and 5 – Beat Skipping:**
 - **Pad 4:** Implements a function to skip backward by 8 beats, allowing DJs to rewind the track and realign beats.
 - **Pad 5:** Implements a function to skip forward by 8 beats, enabling quick navigation and beat scratching effects.

Code Snippets for Drum Pads

Drum Pad 1 & Drum Pad 6 – Loading & Reloading Samples

These pads load a sample (e.g., vocal track or sound effect) when first clicked and allow reloading by double-clicking. Only the first 10 seconds of the loaded sample are played.

```
void DrumPads::DrumPadButton::mouseDoubleClick(const juce::MouseEvent& event)
{
    if (getName() == "Pad1" || getName() == "Pad6")
    {
        if (auto* parent = findParentComponentOfClass<DrumPads>())
            parent->handleDrumPadDoubleClick(getName());
    }
    juce::TextButton::mouseDoubleClick(event);
}
```

And in the parent component:

```
// -- Double-Click Handling --
// Reload vocal samples for Pad1 or Pad6 on double-click.
void DrumPads::handleDrumPadDoubleClick(const juce::String& padName)
{
    if (padName == "Pad1")
    {
        DBG("DrumPad #1 double-clicked: Reload Vocal1 sample");
        vocal1URL = juce::URL();
        launchFileChooser("Select a new track for Vocal #1", [this](const juce::URL& url)
        {
            vocal1URL = url;
            DBG("Reloaded Vocal1 sample: " + vocal1URL.toString(true));
        });
    }
    else if (padName == "Pad6")
    {
        DBG("DrumPad #6 double-clicked: Reload Vocal2 sample");
        vocal2URL = juce::URL();
        launchFileChooser("Select a new track for Vocal #2", [this](const juce::URL& url)
        {
            vocal2URL = url;
            DBG("Reloaded Vocal2 sample: " + vocal2URL.toString(true));
        });
    }
}
```

For the initial onClick loading (if no sample is loaded):

```
drumPadButtons[0]->onClick = [this]
{
    DBG("DrumPad #1 clicked");
    if (vocal1URL.isEmpty())
        launchFileChooser("Select a track for Vocal #1", [this](const juce::URL& url)
        {
            vocal1URL = url;
            DBG("Loaded Vocal1 sample: " + vocal1URL.toString(true));
        });
    else
    {
        if (onVocal1Triggered)
        {
            DBG("Triggering Vocal1 sample playback");
            onVocal1Triggered();
        }
    }
};
```

```
drumPadButtons[5]->onClick = [this]
{
    DBG("DrumPad #6 clicked");
    if (vocal2URL.isEmpty())
        launchFileChooser("Select a track for Vocal #2", [this](const juce::URL& url)
        {
```

```

        vocal2URL = url;
        DBG("Loaded Vocal2 sample: " + vocal2URL.toString(true));
    });
    else
    {
        if (onVocal2Triggered)
        {
            DBG("Triggering Vocal2 sample playback");
            onVocal2Triggered();
        }
    }
};

```

Drum Pad 2 – Bass Drop Effect

Triggers a synthesized bass drop effect (no audio file loaded):

```

drumPadButtons[1]->onClick = [this]
{
    DBG("DrumPad #2 clicked (Bass Drop)");
    if (onBassDropTriggered)
        onBassDropTriggered();
};

```

And in the DJAudioPlayer class, the bass drop effect is processed as follows:

```

// Processes and applies the bass drop effect to the audio block.
void DJAudioPlayer::processBassDrop(const juce::AudioSourceChannelInfo& bufferToFill)
{
    const int numChannels = bufferToFill.buffer->getNumChannels();
    const int numSamples = bufferToFill.numSamples;
    for (int channel = 0; channel < numChannels; ++channel)
    {
        float* writePtr = bufferToFill.buffer->getWritePointer(channel, bufferToFill.startSample);
        for (int i = 0; i < numSamples; ++i)
        {
            double dt = 1.0 / currentSampleRate;
            bassDropElapsed += dt;
            double t = bassDropElapsed / bassDropDuration;
            if (t > 1.0) t = 1.0;
            double currentFreq = bassDropStartFreq * (1.0 - t) + bassDropEndFreq * t;
            double currentAmp = 1.0 - t;
            float sampleValue = static_cast<float>(currentAmp * std::sin(bassDropPhase));
            writePtr[i] += sampleValue;
            bassDropPhase += (2.0 * juce::MathConstants<double>::pi * currentFreq) / currentSampleRate;
            if (bassDropPhase > 2.0 * juce::MathConstants<double>::pi)
                bassDropPhase -= 2.0 * juce::MathConstants<double>::pi;
        }
    }
    if (bassDropElapsed >= bassDropDuration)
        isBassDropActive = false;
}

```

Drum Pad 3 – Kick Roll Effect

Triggers a synthesized kick roll effect:

```
drumPadButtons[2]->onClick = [this]
{
    DBG("DrumPad #3 clicked (Kick Roll)");
    if (onKickRollTriggered)
        onKickRollTriggered();
};
```

And in the DJAudioPlayer class, the kick roll effect is processed as follows:

```
// Processes and applies the kick roll effect to the audio block.
void DJAudioPlayer::processKickRoll(const juce::AudioSourceChannelInfo& bufferToFill)
{
    const int numChannels = bufferToFill.buffer->getNumChannels();
    const int numSamples = bufferToFill.numSamples;
    for (int channel = 0; channel < numChannels; ++channel)
    {
        float* writePtr = bufferToFill.buffer->getWritePointer(channel, bufferToFill.startSample);
        for (int i = 0; i < numSamples; ++i)
        {
            if (kickRollGapCounter > 0)
            {
                kickRollGapCounter--;
                continue;
            }
            float sampleValue = static_cast<float>(kickRollAmplitude * std::sin(kickRollPhase));
            writePtr[i] += sampleValue;
            kickRollPhase += (2.0 * juce::MathConstants<double>::pi * kickRollFrequency) / currentSampleRate;
            if (kickRollPhase > 2.0 * juce::MathConstants<double>::pi)
                kickRollPhase -= 2.0 * juce::MathConstants<double>::pi;
            kickRollAmplitude *= kickRollAmpDecay;
            if (kickRollAmplitude < 0.01)
            {
                kickRollRepetitions++;
                if (kickRollRepetitions >= maxKickRollRepetitions)
                {
                    isKickRollActive = false;
                    break;
                }
            }
            else
            {
                kickRollPhase = 0.0;
                kickRollFrequency = 80.0;
                kickRollAmplitude = 2.0;
                kickRollGapCounter = kickRollGapSamples;
            }
        }
    }
}
```

Drum Pads 4 & 5 – Beat Skipping

These pads enable beat skipping by moving backward or forward by 8 beats.

```
drumPadButtons[3]->onClick = [this]
{
    DBG("DrumPad #4 clicked (Skip Backward)");
    if (onSkipBackwardTriggered)
        onSkipBackwardTriggered();
};
drumPadButtons[4]->onClick = [this]
{
    DBG("DrumPad #5 clicked (Skip Forward)");
    if (onSkipForwardTriggered)
        onSkipForwardTriggered();
};
```

And in the DJAudioPlayer, the skip functions will be implemented as:

```
// Skips forward a given number of beats based on BPM.
void DJAudioPlayer::skipForward(int beats, double bpm)
{
    double offset = beats * (60.0 / bpm);
    double newPos = transportSource.getCurrentPosition() + offset;
    double trackLength = transportSource.getLengthInSeconds();
    if (newPos > trackLength)
        newPos = trackLength;
    setPosition(newPos);
}
// Skips backward a given number of beats based on BPM.
void DJAudioPlayer::skipBackward(int beats, double bpm)
{
    double offset = beats * (60.0 / bpm);
    double newPos = transportSource.getCurrentPosition() - offset;
    if (newPos < 0.0)
        newPos = 0.0;
    setPosition(newPos);
}
```

Jog Wheel

Purpose and Functionality

The jog wheel is essential for precise control of the track's playback position—mimicking the functionality of physical vinyl or CD jog wheels used by DJs.

How It Works

- **Interaction:**
Users click and drag on the jog wheel; the component calculates the change in angle relative to its center.

- **Conversion to Position:**

The angular difference (delta) is normalized by dividing by 2π , yielding a value between 0.0 and 1.0 that represents the relative change in playback position.

- **Callbacks:**

Custom callbacks (**onJogTurn** and **onJogEnd**) are invoked to update both the visual playhead and the underlying audio player's position.

Code Snippet:

```
void JogWheel::mouseDrag(const juce::MouseEvent& event)
{
    if (isDragging)
    {
        if (!isPointInsideWheel(event.position))
            return;
        float currentAngle = computeAngle(event.position);
        float deltaAngle = currentAngle - lastAngle;
        // Adjust deltaAngle if wrapping around.
        if (deltaAngle > juce::MathConstants<float>::pi)
            deltaAngle -= juce::MathConstants<float>::twoPi;
        else if (deltaAngle < -juce::MathConstants<float>::pi)
            deltaAngle += juce::MathConstants<float>::twoPi;
        lastAngle = currentAngle;
        float deltaPosition = deltaAngle / juce::MathConstants<float>::twoPi;
        playheadPosition = juce::jlimit(0.0f, 1.0f, playheadPosition + deltaPosition);
        if (onJogTurn)
            onJogTurn(deltaPosition);
        repaint();
    }
}
```

Three-Band EQ (Low, Mid, High)

Purpose and Functionality

The EQ controls allow the DJ to adjust the audio frequency balance:

- **Low EQ:** Boosts or cuts frequencies below ~200 Hz (bass).
- **Mid EQ:** Adjusts frequencies approximately between 200 Hz and 2 kHz (vocals/instruments). This control was added in the final design to provide more refined sound shaping.
- **High EQ:** Influences frequencies above ~2 kHz (treble), affecting clarity and brightness.

Research Material

I reviewed materials such as [this YouTube video](#) to see how DJs manipulate EQ in live settings. Additionally, I referred to [this guide on EQ frequency bands](#) to understand the specific frequency ranges and their impact on the overall mix.

How It Works

Each EQ control is implemented as a rotary slider. Adjusting these sliders calls functions in the **DJAudioPlayer** class, which update filter coefficients in real time.

Code Snippet:

```
// Sets the low EQ gain and updates filters.
void DJAudioPlayer::setLowGain(float newLowGainDb)
{
    lowGainDb = newLowGainDb;
    updateFilters();
}

// Sets the mid EQ gain and updates filters.
void DJAudioPlayer::setMidGain(float newMidGainDb)
{
    midGainDb = newMidGainDb;
    updateFilters();
}

// Sets the high EQ gain and updates filters.
void DJAudioPlayer::setHighGain(float newHighGainDb)
{
    highGainDb = newHighGainDb;
    updateFilters();
}

// Updates the EQ filters using the current gain settings.
void DJAudioPlayer::updateFilters()
{
    if (currentSampleRate <= 0.0)
        return;
    {
        float frequency = 100.0f;
        float Q = 0.7f;
        float lowLinear = juce::Decibels::decibelsToGain(lowGainDb);
        auto lowCoeffs = juce::dsp::IIR::Coefficients<float>::makeLowShelf(currentSampleRate, frequency, Q,
lowLinear);
        *eqChain.get<0>().state = *lowCoeffs;
    }
    {
        float frequency = 1000.0f;
        float Q = 1.0f;
        float midLinear = juce::Decibels::decibelsToGain(midGainDb);
        auto midCoeffs = juce::dsp::IIR::Coefficients<float>::makePeakFilter(currentSampleRate, frequency, Q,
midLinear);
        *eqChain.get<1>().state = *midCoeffs;
    }
    {
        float frequency = 8000.0f;
        float Q = 0.7f;
        float highLinear = juce::Decibels::decibelsToGain(highGainDb);
        auto highCoeffs = juce::dsp::IIR::Coefficients<float>::makeHighShelf(currentSampleRate, frequency, Q,
highLinear);
        *eqChain.get<2>().state = *highCoeffs;
    }
}
```

Record Button

Purpose and Functionality

The record button allows DJs to capture their live mixes, which is essential for practice, review, and sharing creative sets.

How It Works

- **User Interaction:**
A toggle record button is added to the deck interface (typically on Deck 1). When activated, it starts recording the master output.
- **Recording Process:**
The record button triggers the **onRecordToggled** callback. This callback initiates a **MasterRecorder** instance, which captures the mixed audio and writes it to a WAV file stored on the desktop.

Code Snippet:

```
if (enableRecording)
{
    addAndMakeVisible(recordButton);
    recordButton.onClick = [this]()
    {
        bool recordState = recordButton.getToggleState();
        if (onRecordToggled)
            onRecordToggled(recordState);
    };
}
```

And within the **MasterRecorder** class:

```
// -- startRecording --
// Initiates recording, creates a WAV file on the desktop, and sets up the writer.
bool startRecording(double sampleRate)
{
    if (recording)
        return false;
    auto timestamp = juce::Time::getCurrentTime().formatted("%Y%m%d_%H%M%S");
    recordingFile = juce::File::getSpecialLocation(juce::File::userDesktopDirectory)
        .getChildFile("recording_" + timestamp + ".wav");
    juce::WavAudioFormat wavFormat;
    std::unique_ptr<juce::FileOutputStream> fos(recordingFile.createOutputStream());
    if (fos != nullptr)
    {
        writer.reset(wavFormat.createWriterFor(fos.get(), sampleRate, 2, 16, {}, 0));
        fos.release();
    }
    if (writer != nullptr)
    {
        recording = true;
        DBG("Master recording started at: " << recordingFile.getFullPathName());
    }
}
```

```

    }
    else
    {
        DBG("Failed to start master recording!");
    }
    return recording;
}
// -- stopRecording --
// Stops recording and resets the writer.
void stopRecording()
{
    if (recording)
    {
        writer.reset();
        recording = false;
        DBG("Master recording stopped. File: " << recordingFile.getFullPathName());
    }
}

```

VU Meter

Purpose and Functionality

The VU meter serves both a visual and functional role by displaying real-time audio levels, which helps DJs monitor the balance of their mix and avoid clipping.

How It Works

- **Display:**
The VU meter is comprised of a series of colored blocks that represent the RMS (root-mean-square) levels of the left and right audio channels.
- **Real-Time Updates:**
Audio processing calculates current RMS levels, which are then used to dynamically adjust the intensity and color of the blocks. A timer periodically triggers the **repaint()** method to update the display.

Code Snippet:

```

void VUMeter::paint(juce::Graphics& g)
{
    g.fillAll(juce::Colours::black);
    auto bounds = getLocalBounds().toFloat();
    float halfWidth = bounds.getWidth() * 0.5f;
    const float totalSpacing = blockSpacing * (numBlocks - 1);
    float blockHeight = (bounds.getHeight() - totalSpacing) / static_cast<float>(numBlocks);
    float leftX = bounds.getX() + 4.0f;
    float rightX = bounds.getX() + halfWidth + 4.0f;
    float barWidth = (halfWidth - 8.0f);
    // Scale levels using sensitivity.
    float scaledLeftLevel = juce::jlimit(0.0f, 1.0f, leftLevel * meterSensitivity);
    float scaledRightLevel = juce::jlimit(0.0f, 1.0f, rightLevel * meterSensitivity);
}

```

```

// Draw blocks for both channels.
for (int i = 0; i < numBlocks; ++i)
{
    float fraction = static_cast<float>(i) / static_cast<float>(numBlocks - 1);
    float y = bounds.getBottom() - (i + 1) * blockHeight - i * blockSpacing;
    juce::Rectangle<float> leftBlock(leftX, y, barWidth, blockHeight);
    if (fraction <= scaledLeftLevel)
    {
        g.setColour(getMeterColour(fraction));
        g.fillRect(leftBlock);
    }
    juce::Rectangle<float> rightBlock(rightX, y, barWidth, blockHeight);
    if (fraction <= scaledRightLevel)
    {
        g.setColour(getMeterColour(fraction));
        g.fillRect(rightBlock);
    }
}

// Draw border and a dividing line.
g.setColour(juce::Colours::white);
g.drawRect(bounds, 2.0f);
g.setColour(juce::Colours::grey);
g.drawLine(bounds.getX() + halfWidth, bounds.getY(),
           bounds.getX() + halfWidth, bounds.getBottom(), 2.0f);
}

// -- Timer Callback --
// Repaints the component at regular intervals.
void VUMeter::timerCallback()
{
    repaint();
}

```

Summary of R3

My primary extension in this project is the implementation of **Drum Pads**, a feature that significantly enhances performance capabilities. Based on research—including insights from [this video](#) and other online materials—I designed the drum pads to perform the following functions:

- **Drum Pad 1 & Drum Pad 6:**
 - Load vocal tracks or sound effects via a file chooser.
 - Enable quick reloading through a double-click mechanism.
 - Limit playback to the first 10 seconds, ensuring these pads function as short, triggerable clips.
- **Drum Pad 2:**
 - Triggers a synthesized bass drop effect for dramatic transitions.
- **Drum Pad 3:**
 - Initiates a kick roll effect, providing a rolling kick drum sound.
- **Drum Pads 4 & 5:**

- Enable beat skipping by moving backward (Pad 4) or forward (Pad 5) by 8 beats, facilitating beat alignment and scratching effects.

In addition to the drum pads, I implemented:

- A **Jog Wheel** for precise, rotation-based track positioning.
- **Three-Band EQ Controls** (Low, Mid, High), where the mid control was added based on additional research (see [this video](#) and [this guide](#)) to allow detailed frequency shaping.
- A **Record Button** that enables the DJ to capture live mixes for review and sharing.
- **VU Meters** that provide a visually dynamic and functional display of real-time audio levels.

Collectively, these enhancements deliver a comprehensive and professional DJ interface that not only replicates the functionality of real-world DJ controllers but also introduces advanced interactive features, greatly enriching the live performance experience.
